TECHNICAL REPORT

Enterprise Student Record & Academic Management Platform

Automated CI/CD Pipeline & Kubernetes Orchestration

PROJECT DETAILS

Field	Details
Author	Saiesh Yarramreddy
GitHub	https://github.com/STERDINATOR/DeOps-Mini-Project---Academic-Management-Platform
Report date	October 11, 2026
Environment	Windows / Docker Desktop / Jenkins / Minikube
Technology stack	Python 3.11, Flask 3.0.3, Pytest, Flake8, Docker, Kubernetes

Abstract

This report documents the delivery workflow for a Flask-based academic management platform adapted for a university DevOps assignment. The application manages students, faculty, courses, enrollments, academic marks and credit-weighted GPA. The workflow combines GitHub version control, a six-stage Jenkins pipeline, automated testing and coverage, Docker containerization, and deployment to a three-replica Kubernetes workload running on Minikube. Results in this report are based on the command output and Jenkins console logs supplied during the project work.

> **Implementation note**
>
> The Kubernetes cluster runs locally through Docker Desktop and Minikube. This report does not claim an AWS EC2 deployment. Selected console excerpts are transcribed from the execution logs shared for this project; they are not presented as screenshots from another project.

01 Architecture & GitHub

A focused college portal backed by a Flask service layer and a local in-memory store.

The platform is intentionally simple: a browser interface for academic administration, Flask web routes for pages, REST endpoints for operations, and a service layer that owns validation rules. The database module uses a thread-safe in-memory singleton, avoiding external database setup for this coursework project.



Core modules

Module	Responsibilities
Students	Create, view and delete student records; validate IDs and required details.
Faculty	Maintain faculty records and department information.
Courses	Maintain course credits, semester and assigned faculty.
Enrollments	Validate student/course existence, duplicate enrollment, semester match and the 18-credit cap.
Marks & GPA	Record or update marks, map marks to grade points and calculate credit-weighted GPA.
DevOps assets	Dockerfile, Jenkinsfile, and Kubernetes Deployment/Service manifests are versioned with the source.

Repository

Public GitHub repository: https://github.com/STERDINATOR/DeOps-Mini-Project---Academic-Management-Platform

PROJECT LAYOUT (ABBREVIATED)

```
student-academic-platform/
├── app/              Flask routes, services, models, templates, static files
├── database/         Thread-safe in-memory database
├── tests/            Service, marks/GPA and API tests
├── reports/          JUnit XML test report output
├── k8s/              deployment.yaml and service.yaml
├── Dockerfile
├── Jenkinsfile
├── requirements.txt
└── run.py
```

> **Architecture trade-off**
>
> In-memory storage keeps the application easy to run and test, but records reset when the application process restarts. Persistent storage and authentication are outside the current assignment implementation.

02 Environment Prerequisites

The local Windows toolchain used to build, test, containerize and orchestrate the application.

Jenkins executes on a native Windows workspace and creates a Python 3.11 virtual environment for the pipeline. Docker Desktop supplies the Linux container engine. Minikube provides a local Kubernetes control plane, and kubectl applies the project manifests.

Tool / component	Version or observed state
Python used by Jenkins	Python 3.11.9
Flask	3.0.3
Pytest / pytest-cov	8.2.1 / 5.0.0
Flake8	7.0.0
Docker Engine / CLI	29.5.3; Docker Desktop, Linux engine
Minikube	Installed version 1.39.0; cluster control plane was Running
Kubernetes node	minikube, Ready; cluster Kubernetes version v1.37.0
Git	2.56.0.windows.1 in Jenkins console
Jenkins	Locally running WAR-based Jenkins instance at localhost:8080

Verification commands

WINDOWS POWERSHELL

```
py -3.11 --version
docker version
docker info
minikube status
kubectl get nodes
kubectl config current-context
```

Cluster readiness evidence

VERIFIED OUTPUT SUMMARY

```
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

NAME       STATUS   ROLES           VERSION
minikube   Ready    control-plane   v1.37.0
```

> **Operational dependency**
>
> The Jenkins deployment stage requires Docker Desktop and the Minikube cluster to be available. A build initially stopped when Minikube was Stopped; after starting the cluster, the full pipeline completed successfully.

03 Project Structure & Git Workflow

Incremental changes are recorded in feature branches and integrated into main through pull requests.

The GitHub repository stores application code and DevOps configuration together. The working tree was verified clean on main during local test validation. The foundation and Windows-compatible Jenkins pipeline changes were submitted as separate pull requests.

Relevant project structure

REPOSITORY CONTENTS

```
app/
  models/      student.py, faculty.py, course.py
  routes/      api.py, web.py
  services/    student_service.py
  templates/   dashboard and management pages
  static/      CSS and JavaScript
database/db.py
tests/
  test_student_service.py
  test_marks_mgmt.py
  test_api_endpoints.py
k8s/
  deployment.yaml
  service.yaml
Dockerfile | Jenkinsfile | requirements.txt | run.py
```

Pull requests integrated

Pull request	Purpose	Status
#1 - Add foundation for test report artifacts	Adds reports/.gitkeep so the report folder is tracked; generated XML is ignored.	Merged into main
#2 - Configure Windows-compatible Jenkins CI/CD pipeline	Uses Windows batch commands, Python 3.11 venv, Flake8, Pytest, Docker build and Minikube deployment.	Merged into main

Representative Git workflow

COMMAND PATTERN

```
git switch main
git pull --ff-only origin main
git switch -c feature/jenkins-ci
# edit and validate Jenkinsfile
git add Jenkinsfile
git commit -m "ci: configure Windows-compatible Jenkins pipeline"
git push -u origin feature/jenkins-ci
```

> **Traceability**
>
> The Jenkins build log identifies the main-branch merge commit for PR #2 and checks out source from the public GitHub repository before executing the pipeline.

04 Jenkins CI/CD Automation

The successful run executes six stages and stops when a required quality or deployment command fails.

The Declarative Pipeline is stored in the repository root as Jenkinsfile. Jenkins checks out main, creates a Python 3.11 virtual environment, installs pinned dependencies, runs Flake8 and Pytest, builds the Docker image, then loads that image into Minikube and applies the Kubernetes manifests.

Stage	Automated action	Final run
1. SCM Checkout	Checkout source from GitHub; check Git availability.	Passed
2. Environment Setup	Create .venv with Python 3.11 and install requirements.	Passed
3. Code Linting	Run Flake8 against app/, database/, tests/ and run.py.	Passed
4. Automated Tests	Run 40 tests with coverage and write reports/test-results.xml; publish JUnit.	Passed
5. Docker Build	Build student-academic-platform:v1.0.0.	Passed
6. Kubernetes Deployment	Check Minikube, load image, apply manifests and wait for rollout.	Passed

Selected successful Jenkins console excerpts

BUILD AND QUALITY SUMMARY

```
platform win32 -- Python 3.11.9
collected 40 items
============================== 40 passed ==============================
TOTAL coverage: 87%

SUCCESS: All six DevOps pipeline stages completed.
Finished: SUCCESS
```

DEPLOYMENT SUMMARY FROM FINAL RUN

```
deployment "student-deployment" successfully rolled out

student-deployment-7f76fc9999-9dkln   1/1   Running
student-deployment-7f76fc9999-twhjw   1/1   Running
student-deployment-7f76fc9999-x8jm4   1/1   Running

student-service   NodePort   5000:30090/TCP
```

Troubleshooting history

Early builds exposed environment issues: Python 3.11 was initially unavailable to Jenkins, Docker Desktop was temporarily unreachable, and Minikube was stopped for one run. After correcting these prerequisites, the final reported build finished SUCCESS.

05 Code Quality & Testing

Automated checks exercise API behavior and domain rules before the image is built.

Metric	Observed result
Test cases collected	40
Passed / failed	40 passed / 0 failed
Total statement coverage	87%
Flake8	Completed with no reported findings
JUnit XML	reports/test-results.xml generated and recorded by Jenkins

Selected coverage by module

Module	Coverage
app/routes/web.py	100%
app/services/student_service.py	96%
app/__init__.py	96%
app/models/faculty.py	81%
app/models/student.py	78%
app/routes/api.py	75%
app/models/course.py	74%

Docker image and runtime

The Dockerfile uses python:3.11-slim, installs curl and the pinned requirements, copies the application, creates a non-root appuser, exposes port 5000, and configures a health check against /api/health. The required image tag is student-academic-platform:v1.0.0.

DOCKER COMMANDS USED

```
docker build -t student-academic-platform:v1.0.0 .
docker run -d --name student-platform -p 5000:5000 student-academic-platform:v1.0.0
docker ps
docker logs student-platform
Invoke-RestMethod http://localhost:5000/api/health
```

OBSERVED DOCKER VERIFICATION

```
CONTAINER: student-platform
IMAGE:     student-academic-platform:v1.0.0
STATUS:    Up (healthy)
PORTS:     0.0.0.0:5000 -> 5000/tcp

HEALTH RESPONSE
service: Enterprise Student Record & Academic Management Platform
status:  UP
version: 1.0.0
```

06 Kubernetes Orchestration & Self-Healing

Minikube runs the application as a Deployment with three replicas behind a NodePort Service.

Deployment configuration

Setting	Configured value
Deployment	student-deployment
Pod label / selector	app: student-platform
Replicas	3
Container image	student-academic-platform:v1.0.0
Container port	5000
Readiness and liveness probes	HTTP GET /api/health on port 5000
Resources	Requests: 128Mi / 100m; limits: 256Mi / 250m
Service	student-service, NodePort 30090 -> container/service port 5000

Deployment verification commands

KUBERNETES COMMANDS

```
minikube status
kubectl get nodes
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl rollout status deployment/student-deployment --timeout=180s
kubectl get pods -l app=student-platform -o wide
kubectl get services
```

FINAL JENKINS DEPLOYMENT OUTPUT

```
deployment "student-deployment" successfully rolled out

NAME                          READY   STATUS
student-deployment-7f76fc9999-9dkln   1/1     Running
student-deployment-7f76fc9999-twhjw   1/1     Running
student-deployment-7f76fc9999-x8jm4   1/1     Running

student-service   NodePort   5000:30090/TCP
```

Self-healing demonstration

The self-healing exercise was performed by manually deleting one Pod selected from the app=student-platform label. The Deployment controller recreated a replacement Pod, and the desired count returned to three healthy replicas. This demonstrates Kubernetes reconciliation of the actual replica count.

SELF-HEALING TEST PROCEDURE

```
kubectl get pods -l app=student-platform -o wide
$podToDelete = kubectl get pods -l app=student-platform -o jsonpath="{.items[0].metadata.name}"
kubectl delete pod $podToDelete
kubectl get pods -l app=student-platform --watch
# Ctrl+C stops the watch only; it does not stop Kubernetes.
```

07 REST API Validation & Conclusion

Health checks and academic workflows confirm that the deployed service responds through the expected interface.

Health endpoint

VERIFIED RESPONSE SHAPE

```
GET /api/health

{
  "status": "UP",
  "service": "Enterprise Student Record & Academic Management Platform",
  "version": "1.0.0"
}
```

REST endpoints

Area	Endpoint examples
Health	GET /api/health
Students	POST /api/students; GET /api/students/<student_id>; DELETE /api/students/<student_id>
Faculty	POST /api/faculty; GET /api/faculty/<faculty_id>
Courses	POST /api/courses; GET /api/courses; GET /api/courses/<course_id>
Enrollments	POST /api/enroll; GET /api/students/<student_id>/enrollments
Marks & GPA	POST /api/marks; GET /api/students/<student_id>/gpa

Academic rules represented in the application

- Student, faculty and course identifiers must be unique; required fields and positive semesters/credits are validated.
- Enrollment rejects unknown students/courses, duplicate course registration, semester mismatches, and totals above 18 credits per semester.
- Marks are restricted to 0-100 and may be recorded only for an enrolled student/course pair; existing marks can be updated.
- Grade points: 90-100=10, 80-89=9, 70-79=8, 60-69=7, 50-59=6, 40-49=5, 0-39=0. GPA is weighted by course credits and rounded to two decimal places.

Conclusion

The Student Record & Academic Management Platform demonstrates a complete local DevOps delivery chain: version-controlled source, linting and automated tests in Jenkins, a tagged Docker image, and a three-replica Kubernetes Deployment exposed through NodePort 30090. The final Jenkins console log reports SUCCESS, 40 of 40 passing tests, 87% overall coverage, and a successful Kubernetes rollout.

Known limitations and next improvements

The datastore is in-memory and is reset by process restarts. Authentication and persistent storage are not implemented. The Docker container currently starts the Flask development server. Future improvements should add a persistent database, access control, and a production WSGI server if the application is used beyond the coursework environment.

Repository: https://github.com/STERDINATOR/DeOps-Mini-Project---Academic-Management-Platform